

EPAM Java Developer Interview Prep Guide

Comprehensive Q&A — Core Java, Collections, Streams, Concurrency, Spring & Live Coding

Prepared for: Saturday, July 18, 2026 · 4:00 PM IST · Primary skill: Java

1. Collections Framework

Q: Difference between HashMap and TreeMap?

A: HashMap stores entries in no guaranteed order and offers $O(1)$ average time for get/put by hashing keys into buckets. TreeMap keeps keys in sorted order (natural ordering or a Comparator) using a Red-Black tree internally, giving $O(\log n)$ for get/put but letting you do range queries and ordered iteration. Use HashMap when you just need fast lookup; use TreeMap when you need sorted keys or range operations like `firstKey()/floorKey()`.

Q: Difference between HashMap, LinkedHashMap, and TreeMap in terms of ordering?

A: HashMap: no order guarantee. LinkedHashMap: preserves insertion order (or access order if configured, useful for LRU caches). TreeMap: sorted key order. Pick LinkedHashMap when you want predictable iteration order without the cost of sorting.

Q: How does HashMap work internally?

A: Each key's `hashCode()` is passed through a hash-spreading function and mapped to a bucket index in an internal array. Each bucket holds a linked list of entries with the same bucket index (collisions). Since Java 8, if a single bucket's linked list grows beyond a threshold (8) and the table is large enough, that bucket converts to a balanced tree (red-black tree) so worst-case lookup becomes $O(\log n)$ instead of $O(n)$. The map resizes (doubles capacity) when the number of entries exceeds capacity * loadFactor (default load factor 0.75).

Q: What happens if two keys have the same hashCode() but are not equal?

A: They land in the same bucket (a hash collision), but since `equals()` returns false, both are stored separately as distinct entries in that bucket's list/tree. Lookup uses `hashCode()` to find the bucket, then `equals()` to find the exact match.

Q: ArrayList vs LinkedList vs Vector?

A: ArrayList is backed by a resizable array — $O(1)$ random access, $O(n)$ insert/delete in the middle, not synchronized. LinkedList is a doubly linked list — $O(1)$ insert/delete at known positions (like head/tail via Deque methods), $O(n)$ random access, more memory overhead per element. Vector is essentially a legacy synchronized ArrayList — rarely used today; prefer ArrayList with explicit synchronization or `java.util.concurrent` classes instead.

Q: HashSet vs TreeSet vs LinkedHashSet?

A: HashSet: backed by a HashMap internally, no order, $O(1)$ average add/contains. LinkedHashSet: preserves insertion order. TreeSet: sorted order, backed by a TreeMap, $O(\log n)$ operations, supports NavigableSet methods like `higher()/lower()`.

Q: ConcurrentHashMap vs Collections.synchronizedMap?

A: `Collections.synchronizedMap` wraps a map and synchronizes every method call on a single lock — so only one thread can access the map at all, even for unrelated keys, and iteration still needs manual external synchronization. `ConcurrentHashMap` uses fine-grained internal locking (bucket-level in modern versions) so multiple threads can read/write different segments concurrently without blocking each other, and its iterators are weakly consistent (don't throw `ConcurrentModificationException`). Prefer `ConcurrentHashMap` in concurrent code.

Q: Comparable vs Comparator?

A: Comparable is implemented by the class itself (`compareTo` method) and defines a single 'natural ordering' — e.g., `Employee` implements Comparable to sort by id by default. Comparator is a separate strategy object (`compare` method) that lets you define multiple, external, interchangeable sort orders without modifying the class — e.g., `Comparator.comparing(Employee::getName).reversed()`.

```
// Comparable example
class Employee implements Comparable<Employee> {
    int id; String name;
    public int compareTo(Employee o) { return Integer.compare(this.id, o.id); }
}

// Comparator example (no need to touch the class)
list.sort(Comparator.comparing(Employee::getName).reversed());
```

» TIP: HashMap internals and Comparable vs Comparator are asked almost every round — know them cold, don't just skim.

2. Java 8+ and Streams (very likely a live coding task)

Q: Filter employees from a given city and sort them in reverse alphabetical order by name (classic EPAM live question).

A: Use stream(), filter() on city, sorted() with a reversed Comparator, then collect to a List.

```
List<Employee> result = employees.stream()
    .filter(e -> e.getCity().equalsIgnoreCase("Noida"))
    .sorted(Comparator.comparing(Employee::getName).reversed())
    .collect(Collectors.toList());
```

Q: Find the first non-repeating character in a string using streams.

A: Build a LinkedHashMap of character counts to preserve order, then stream over entrySet and find the first entry with count == 1.

```
public static Character firstNonRepeating(String s) {
    Map<Character, Long> counts = s.chars()
        .mapToObj(c -> (char) c)
        .collect(Collectors.groupingBy(c -> c, LinkedHashMap::new, Collectors.counting()));

    return counts.entrySet().stream()
        .filter(e -> e.getValue() == 1)
        .map(Map.Entry::getKey)
        .findFirst()
        .orElse(null);
}
```

Q: Find the first repeating character in a string.

A: Track a Set of seen characters while iterating; the first character you encounter that's already in the set is the first repeating one.

```
public static Character firstRepeating(String s) {
    Set<Character> seen = new HashSet<>();
    for (char c : s.toCharArray()) {
        if (!seen.add(c)) return c;
    }
    return null;
}
```

Q: Group a list of objects by a field and count them (Collectors.groupingBy).

```
Map<String, Long> countByCity = employees.stream()
    .collect(Collectors.groupingBy(Employee::getCity, Collectors.counting()));
```

Q: Convert a List to a Map using streams.

```
Map<Integer, String> idToName = employees.stream()
    .collect(Collectors.toMap(Employee::getId, Employee::getName));
```

Q: What's the difference between `map()` and `flatMap()`?

A: `map()` transforms each element 1-to-1 (e.g., `Employee -> String name`). `flatMap()` is used when each element itself produces a stream (e.g., a `List<>` or an `Employee` with a list of phone numbers) — it flattens those nested streams into a single stream, avoiding a `Stream<>`.

Q: What is the difference between intermediate and terminal operations, and why does laziness matter?

A: Intermediate operations (`filter`, `map`, `sorted`) are lazy — they just build up a pipeline and don't execute anything by themselves. A terminal operation (`collect`, `forEach`, `reduce`, `count`) triggers actual traversal of the source, executing all intermediate steps in a single pass. This laziness lets the JVM optimize and avoid unnecessary work, and means a stream can only be consumed once.

Q: When would you use `Optional`, and how do you use it correctly?

A: `Optional` expresses 'this method might not return a value' explicitly in the return type, avoiding null checks scattered everywhere and `NullPointerException`s. Use `orElse()`, `orElseGet()`, or `map()/ifPresent()` to work with it instead of calling `.get()` directly, which defeats the purpose. Don't use `Optional` for fields or method parameters — only as a return type.

Q: What are functional interfaces? Name the core ones.

A: A functional interface has exactly one abstract method and can be implemented with a lambda. Core ones: `Function` (takes `T`, returns `R`), `Predicate` (takes `T`, returns `boolean`), `Supplier` (takes nothing, returns `T`), `Consumer` (takes `T`, returns nothing), and `BiFunction/BinaryOperator` for two-argument variants.

Q: What's a method reference and when do you use it?

A: A method reference (e.g., `Employee::getName`) is shorthand for a lambda that just calls an existing method — cleaner than writing `e -> e.getName()`. Four forms: static (`ClassName::staticMethod`), instance of a particular object (`obj::method`), instance of an arbitrary object of a type (`ClassName::instanceMethod`), and constructor (`ClassName::new`).

» TIP: For the coding round, actually type these out in an IDE before your interview rather than just reading — muscle memory matters more than recognition here.

3. Core Java & OOP Design Principles

Q: Explain the SOLID principles briefly.

A: S — Single Responsibility: a class should have one reason to change. O — Open/Closed: open for extension, closed for modification (use abstraction/polymorphism instead of editing existing code). L — Liskov Substitution: subtypes must be substitutable for their base type without breaking correctness. I — Interface Segregation: prefer many small, specific interfaces over one large general-purpose interface. D — Dependency Inversion: depend on abstractions (interfaces), not concrete implementations — this is the principle behind Spring's dependency injection.

Q: What is the Liskov Substitution Principle, with an example violation?

A: If class `Bird` has a `fly()` method and class `Ostrich` extends `Bird` but throws an exception in `fly()`, that violates LSP — code that works with a `Bird` reference breaks when given an `Ostrich`. Fix: don't force `Ostrich` to extend a `Bird` that assumes flight; separate `FlyingBird` from `Bird`, or use composition instead of inheritance.

Q: Why doesn't Java support multiple inheritance through classes, and how do interfaces solve related problems?

A: Multiple class inheritance creates the 'diamond problem' — if two parent classes define the same method differently, the compiler can't unambiguously decide which implementation to inherit. Java avoids this by

allowing a class to extend only one class, but implement multiple interfaces. Since Java 8, interfaces can have default methods, which reintroduces a limited diamond problem — resolved by requiring the implementing class to explicitly override the conflicting method if two interfaces provide the same default method.

Q: equals() and hashCode() contract — why must you override both together?

A: If two objects are equal according to equals(), they must return the same hashCode(); otherwise hash-based collections (HashMap, HashSet) will fail to find an object you just inserted, because they use hashCode() to pick the bucket and equals() to confirm the match within that bucket. Overriding one without the other breaks this contract silently.

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Employee)) return false;
    Employee e = (Employee) o;
    return id == e.id && Objects.equals(name, e.name);
}
@Override
public int hashCode() { return Objects.hash(id, name); }
```

Q: How do you design an immutable class in Java?

A: Make the class final (prevent subclassing), make all fields private and final, don't provide setters, initialize everything through the constructor, and if a field is a mutable object (like a List or Date), return a defensive copy from any getter instead of the original reference.

Q: Why is String immutable in Java?

A: Security (Strings are used for class names, file paths, network connections — mutation could cause serious bugs), thread-safety (immutable objects are inherently safe to share across threads), and it enables the String pool, which lets the JVM safely reuse identical String literals to save memory.

Q: Abstract class vs Interface — especially post Java 8 default methods?

A: An abstract class can hold state (instance fields), constructors, and a mix of abstract and concrete methods, but a class can extend only one abstract class. An interface traditionally only declared method signatures, but since Java 8 it can have default and static methods with implementations; still, a class can implement many interfaces. Use an abstract class when subclasses share common state/behavior; use an interface to define a capability/contract multiple unrelated classes can adopt.

4. Multithreading & Concurrency

Q: What does the volatile keyword actually guarantee?

A: It guarantees visibility (a write to a volatile variable by one thread is immediately visible to other threads reading it) and prevents instruction reordering around it by the compiler/JIT. It does NOT guarantee atomicity — for example, count++ on a volatile int is still not thread-safe because increment is read-modify-write, three separate steps.

Q: What does synchronized guarantee, and how does it differ from volatile?

A: synchronized provides both mutual exclusion (only one thread can hold the monitor/lock at a time) and visibility (changes made inside a synchronized block become visible to the next thread that acquires the same lock). volatile only gives visibility for a single variable, with no mutual exclusion — so use synchronized (or java.util.concurrent locks) when multiple threads need to perform compound actions on shared state.

Q: Explain the Java Memory Model and the 'happens-before' relationship.

A: The JMM defines rules for how and when writes by one thread become visible to reads by another thread, since each thread may cache variables locally for performance. 'Happens-before' is a partial ordering: if action A happens-before action B, then A's effects are guaranteed visible to B. Examples that establish happens-before: a write to a volatile field happens-before every subsequent read of it; unlocking a monitor happens-before a later thread locking the same monitor; and a thread's actions before calling Thread.start() happen-before the started thread's actions.

Q: Runnable vs Callable?

A: Runnable's run() method returns nothing and cannot throw a checked exception. Callable's call() method returns a value V and can throw checked exceptions, making it useful with ExecutorService.submit() when you need a Future result back.

Q: What is an ExecutorService and why use it instead of creating raw Threads?

A: ExecutorService manages a pool of reusable worker threads, so you avoid the overhead of creating a new OS thread per task and avoid unbounded thread creation. You submit tasks (Runnable/Callable) and the framework schedules them onto the pool, giving you Future objects to track completion/results, plus lifecycle control (shutdown, awaitTermination).

```
ExecutorService executor = Executors.newFixedThreadPool(4);
Future<Integer> future = executor.submit(() -> computeSomething());
Integer result = future.get(); // blocks until done
executor.shutdown();
```

Q: What is a race condition, and how do you prevent one?

A: A race condition occurs when the correctness of a program depends on the relative timing of multiple threads accessing shared mutable state without proper coordination. Prevent it with synchronized blocks/methods, java.util.concurrent locks (ReentrantLock), atomic classes (AtomicInteger, AtomicReference), or by avoiding shared mutable state entirely (immutability, thread confinement).

5. Spring / Spring Boot Essentials

Q: What is Dependency Injection and why does Spring use it?

A: Dependency Injection means a class receives its dependencies (collaborator objects) from an external source rather than creating them itself — usually via constructor injection. This decouples classes from concrete implementations (Dependency Inversion principle), makes unit testing easy (you can inject mocks), and lets Spring's IoC container manage object creation and lifecycle centrally.

```
@Service
public class OrderService {
    private final PaymentGateway paymentGateway; // abstraction, not concrete class

    // Constructor injection (preferred over field injection)
    public OrderService(PaymentGateway paymentGateway) {
        this.paymentGateway = paymentGateway;
    }
}
```

Q: @Component vs @Service vs @Repository vs @Controller — are they functionally different?

A: All four are specializations of @Component and get picked up by Spring's component scanning the same way. They mainly differ semantically/for tooling: @Service marks business logic classes, @Repository marks data-access classes and additionally enables automatic translation of persistence exceptions into Spring's DataAccessException hierarchy, and @Controller/@RestController mark web layer classes and enable request-mapping behavior.

Q: Singleton vs Prototype bean scope in Spring?

A: Singleton (the default) means Spring creates exactly one instance of the bean per container and shares it everywhere it's injected. Prototype means a new instance is created every time the bean is requested/injected. Use prototype for stateful, non-thread-safe beans; singleton for stateless services, which is the common case.

Q: How would you build a simple REST GET endpoint in Spring Boot?

```
@RestController
@RequestMapping("/api/employees")
public class EmployeeController {

    private final EmployeeService service;
    public EmployeeController(EmployeeService service) { this.service = service; }

    @GetMapping("/{id}")
    public ResponseEntity<Employee> getEmployee(@PathVariable Long id) {
        Employee emp = service.findById(id);
        return ResponseEntity.ok(emp);
    }
}
```

Q: What is (de)serialization, and where does it show up in a Spring REST app?

A: Serialization converts a Java object into a transportable format like JSON; deserialization converts JSON back into a Java object (POJO). In Spring Boot, this happens automatically via Jackson when a controller method returns an object (serialized to the HTTP response body) or accepts a @RequestBody parameter (deserialized from the incoming request body) — no manual parsing needed.

6. More Collections Questions

Q: What is the difference between Iterator and ListIterator?

A: Iterator can traverse any Collection (List, Set, Map's keySet/values) only forward, and supports remove() only. ListIterator works only on Lists but can traverse both forward and backward, supports remove(), set() (replace current element), and add(), and also gives you the current index via nextIndex()/previousIndex().

Q: What is ConcurrentModificationException and when does it occur?

A: It's thrown when a Collection is structurally modified (add/remove) while being iterated with a regular Iterator, outside of the iterator's own remove() method. Internally, most collections keep a modCount; the iterator checks it on every next() call and fails fast if it changed. Fix: use iterator.remove(), a CopyOnWriteArrayList/ConcurrentHashMap for concurrent access, or collect items to remove separately and remove them after the loop.

```
// Wrong - throws ConcurrentModificationException
for (String s : list) {
    if (s.equals("x")) list.remove(s);
}

// Correct
Iterator<String> it = list.iterator();
while (it.hasNext()) {
    if (it.next().equals("x")) it.remove();
}
```

Q: Fail-fast vs fail-safe iterators?

A: Fail-fast iterators (ArrayList, HashMap, HashSet) throw ConcurrentModificationException if the underlying collection is modified during iteration, since they operate directly on the collection. Fail-safe iterators (CopyOnWriteArrayList, ConcurrentHashMap) iterate over a snapshot or a version tolerant of concurrent

changes, so no exception is thrown, though the iteration may not reflect the very latest changes.

Q: How would you make a List/Map immutable?

A: Use `List.of(...)`, `Map.of(...)` (Java 9+) for a truly immutable collection, or `Collections.unmodifiableList(list)` which wraps an existing list — but note the wrapper only blocks structural changes through itself; the original underlying list can still be mutated directly and that change will show through the wrapper.

Q: What is the difference between Queue, Deque, and Stack in the Collections framework?

A: Queue is FIFO (offer/poll/peek). Deque (double-ended queue, e.g., `ArrayDeque`) supports insertion/removal at both ends and can be used as either a Queue or a Stack, which is why `ArrayDeque` is now preferred over the legacy Stack class. The legacy Stack class extends `Vector`, is synchronized (slower), and is generally considered outdated.

Q: PriorityQueue — how does it order elements, and what's a common use case?

A: `PriorityQueue` is a min-heap by default (natural ordering via `Comparable`, or a `Comparator` you supply) — `poll()` always returns the smallest element first. Common use cases: Dijkstra's algorithm, task scheduling by priority, finding the k largest/smallest elements in a stream of data.

```
PriorityQueue<Integer> minHeap = new PriorityQueue<>();
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Comparator.reverseOrder());
minHeap.addAll(List.of(5, 1, 8, 3));
minHeap.poll(); // returns 1
```

Q: What is the difference between Array and ArrayList?

A: An array has fixed size once created and can hold primitives directly; an `ArrayList` is resizable (grows automatically, typically by 1.5x when full) but only holds objects (autoboxing wraps primitives), and comes with the full Collections API (sort, contains, streams, etc.) that a plain array doesn't have.

Q: How would you remove duplicates from a List while preserving order?

```
List<Integer> deduped = new ArrayList<>(new LinkedHashSet<>(list));
// or via streams
List<Integer> deduped2 = list.stream().distinct().collect(Collectors.toList());
```

Q: What is the load factor in a HashMap and why is the default 0.75?

A: Load factor determines when the map resizes: resize happens when `size > capacity * loadFactor`. 0.75 is a time/space trade-off Java's designers chose — a higher load factor (e.g., 0.9) saves memory but increases collision chance and slows lookups; a lower one (e.g., 0.5) speeds up lookups but wastes more memory and resizes more often.

7. More Streams Questions

Q: Sort a list of objects by multiple fields (e.g., first by city, then by name).

```
List<Employee> sorted = employees.stream()
    .sorted(Comparator.comparing(Employee::getCity)
        .thenComparing(Employee::getName))
    .collect(Collectors.toList());
```

Q: Find the sum/average of a numeric field across a list of objects.

```
double totalSalary = employees.stream()
    .mapToDouble(Employee::getSalary)
    .sum();

OptionalDouble avgSalary = employees.stream()
    .mapToDouble(Employee::getSalary)
```

```
.average();
```

Q: Partition a list into two groups based on a condition (e.g., salary above/below a threshold).

```
Map<Boolean, List<Employee>> partitioned = employees.stream()
    .collect(Collectors.partitioningBy(e -> e.getSalary() > 50000));

List<Employee> highEarners = partitioned.get(true);
```

Q: Join a list of Strings into a single comma-separated String.

```
String joined = names.stream().collect(Collectors.joining(", ", "[", "]"));
// or simply
String joined2 = String.join(", ", names);
```

Q: Find the employee with the maximum salary using streams.

```
Optional<Employee> topEarner = employees.stream()
    .max(Comparator.comparing(Employee::getSalary));
```

Q: What is the difference between findFirst() and findAny()?

A: findFirst() always returns the first element encountered in encounter order, which matters for ordered streams (like those from a List). findAny() makes no such guarantee and may return any matching element — it exists mainly for parallel streams, where it can be more efficient since it doesn't need to respect ordering across threads.

Q: What does reduce() do? Give an example computing a factorial-style aggregation.

A: reduce() combines stream elements into a single result using a BinaryOperator, starting from an optional identity value. It's a general-purpose way to do sums, products, string concatenation, or any custom fold operation.

```
int product = Stream.of(1, 2, 3, 4, 5)
    .reduce(1, (a, b) -> a * b); // 120
```

Q: What is a parallel stream, and when should you avoid it?

A: stream().parallel() splits the source into chunks processed concurrently across the ForkJoinPool's common pool, which can speed up CPU-heavy operations on large datasets. Avoid it for small datasets (overhead exceeds benefit), when the operations aren't stateless/associative, when order matters and you haven't accounted for it, or when the lambda has side effects on shared mutable state (introduces race conditions).

Q: How do you convert a stream back to an array, a Set, or a specific List implementation?

```
Integer[] arr = list.stream().toArray(Integer[]::new);
Set<Integer> set = list.stream().collect(Collectors.toSet());
LinkedList<Integer> linked = list.stream()
    .collect(Collectors.toCollection(LinkedList::new));
```

8. Classes & Objects — Fundamentals

Q: What is the difference between a class and an object?

A: A class is a blueprint/template that defines fields (state) and methods (behavior). An object is a concrete instance of that class created at runtime via the new keyword, occupying its own memory on the heap with its own copy of instance fields, while sharing the same method code defined in the class.

Q: What happens in memory when you write 'Employee e = new Employee();'?

A: The JVM allocates memory on the heap for a new Employee object, initializes its fields to default values, runs the constructor to set actual values, and returns a reference to that heap memory, which is stored in the local variable 'e' on the stack. 'e' itself just holds the reference (like an address), not the object.

Q: What is constructor overloading, and why use 'this()' to chain constructors?

A: Constructor overloading means defining multiple constructors with different parameter lists so an object can be created in different ways. Using `this(...)` inside one constructor to call another avoids duplicating initialization logic across constructors.

```
class Employee {
    int id; String name; String city;

    Employee(int id, String name) {
        this(id, name, "Unknown"); // chain to the 3-arg constructor
    }
    Employee(int id, String name, String city) {
        this.id = id; this.name = name; this.city = city;
    }
}
```

Q: Static vs instance members — what's the practical difference?

A: Static fields/methods belong to the class itself — one shared copy exists regardless of how many objects are created, accessed via `ClassName.member`. Instance fields/methods belong to each individual object — every object gets its own copy of instance fields. Use static for shared counters, utility/helper methods, or constants; use instance members for per-object state.

Q: What is the difference between method overloading and method overriding?

A: Overloading: same method name, different parameter lists, within the same class — resolved at compile time (static polymorphism). Overriding: a subclass redefines a method with the exact same signature as its superclass — resolved at runtime based on the actual object type (dynamic/runtime polymorphism), and requires the `@Override` annotation as good practice to catch signature mismatches at compile time.

Q: Explain the four pillars of OOP with a one-line example each.

A: Encapsulation: keeping fields private and exposing controlled access via getters/setters, so internal state can't be corrupted directly. Abstraction: exposing only relevant behavior through interfaces/abstract classes while hiding implementation detail. Inheritance: a subclass reuses and extends fields/behavior from a superclass. Polymorphism: the same method call behaves differently depending on the actual runtime object (overriding) or the arguments passed (overloading).

Q: What is object cloning, and shallow vs deep copy?

A: Cloning creates a copy of an object, typically via implementing `Cloneable` and overriding `clone()`. A shallow copy duplicates the object itself but copies references to any nested mutable objects as-is (so both copies share the same nested object) — the default `Object.clone()` behavior. A deep copy also recursively clones nested mutable objects so the two copies are fully independent.

9. Inheritance — Deep Dive

Q: What is the 'is-a' vs 'has-a' relationship, and how does it decide inheritance vs composition?

A: Inheritance models an 'is-a' relationship — a Car is-a Vehicle, so Car extends Vehicle makes sense. Composition models a 'has-a' relationship — a Car has-a Engine, so Car should hold an Engine field rather than extend an Engine class. A common design guideline is 'favor composition over inheritance' because composition is more flexible and avoids tightly coupling classes through a rigid hierarchy.

Q: What is the order of constructor execution in an inheritance chain?

A: When a subclass object is created, Java first implicitly (or explicitly via `super(...)`) calls the superclass constructor before executing the subclass constructor's own body — this guarantees the parent's state is fully initialized before the child adds its own. This chains all the way up to `Object`'s constructor.

```

class Animal {
    Animal() { System.out.println("Animal constructor"); }
}
class Dog extends Animal {
    Dog() {
        super(); // implicit if omitted, but always runs first
        System.out.println("Dog constructor");
    }
}
// new Dog() prints: "Animal constructor" then "Dog constructor"

```

Q: Can a constructor be inherited? Can it be overridden?

A: No — constructors are not inherited and cannot be overridden, because they're tied to the exact class they're declared in and are named after that class. A subclass must define its own constructors (or rely on the compiler-generated no-arg one), though it can call a superclass constructor explicitly via `super(...)`.

Q: Explain method overriding rules: what can and can't change?

A: The method name and parameter list must match exactly. The return type must be the same or a covariant (subtype) return type. Access modifier can't be more restrictive than the parent's (can widen, e.g. protected to public, but not narrow). The overriding method can't throw new or broader checked exceptions than the overridden method, though it can throw fewer or narrower ones.

Q: What is the 'diamond problem' and how does Java handle it with interfaces?

A: The diamond problem arises when a class could inherit conflicting implementations of the same method from two different parents (e.g., two interfaces both providing the same default method). Java avoids ambiguity for classes by disallowing multiple class inheritance entirely. For interfaces with default methods, if two interfaces provide the same default method, the implementing class is forced to explicitly override that method and can call a specific parent's version using `InterfaceName.super.methodName()`.

```

interface A { default void greet() { System.out.println("Hello from A"); } }
interface B { default void greet() { System.out.println("Hello from B"); } }

class C implements A, B {
    // Must override to resolve the conflict
    public void greet() {
        A.super.greet(); // explicitly choose A's version
    }
}

```

Q: What is the difference between 'super' used for constructors vs for methods/fields?

A: `super(...)` as the first statement in a constructor calls the immediate superclass's constructor. `super.methodName()` calls the superclass's version of an overridden method (useful when you want to extend, not fully replace, parent behavior). `super.fieldName` accesses a superclass field that's been shadowed by a field of the same name in the subclass.

Q: Write pseudocode for a class hierarchy modeling Shape -> Circle/Rectangle with polymorphic area calculation.

```

ABSTRACT CLASS Shape:
    ABSTRACT METHOD area() -> double

CLASS Circle EXTENDS Shape:
    FIELD radius
    CONSTRUCTOR(radius): set this.radius
    OVERRIDE area(): RETURN PI * radius * radius

CLASS Rectangle EXTENDS Shape:
    FIELD width, height

```

```

    CONSTRUCTOR(width, height): set fields
    OVERRIDE area(): RETURN width * height

```

```

MAIN:
    shapes = LIST[Circle(5), Rectangle(4, 6)]
    FOR each shape IN shapes:
        PRINT shape.area()    // calls the correct override at runtime
                               // (this is runtime polymorphism)

```

Q: Real Java version of the same Shape/Circle/Rectangle hierarchy.

```

abstract class Shape {
    abstract double area();
}

class Circle extends Shape {
    private final double radius;
    Circle(double radius) { this.radius = radius; }
    @Override double area() { return Math.PI * radius * radius; }
}

class Rectangle extends Shape {
    private final double width, height;
    Rectangle(double width, double height) { this.width = width; this.height = height; }
    @Override double area() { return width * height; }
}

public class Main {
    public static void main(String[] args) {
        List<Shape> shapes = List.of(new Circle(5), new Rectangle(4, 6));
        for (Shape s : shapes) {
            System.out.println(s.area()); // dynamic dispatch picks the right area()
        }
    }
}

```

Q: What is dynamic method dispatch (runtime polymorphism), and how does the JVM implement it?

A: Dynamic dispatch means the JVM decides which overridden method implementation to call based on the actual runtime type of the object, not the reference type used in the code — e.g., a Shape reference pointing to a Circle object calls Circle's area(). Internally, the JVM uses a virtual method table (vtable) per class, so the call is resolved by looking up the actual object's class's vtable entry at invocation time, not at compile time.

Q: Pseudocode: a small inheritance example demonstrating why composition can be safer than inheritance.

```

// Problematic: inheritance couples Stack to List's full API
CLASS Stack EXTENDS ArrayList:
    // inherits add(index, item), remove(index), etc. -
    // callers can break stack discipline by calling those directly

// Safer: composition, expose only what a Stack should allow
CLASS Stack:
    FIELD items = new ArrayList()
    METHOD push(item): items.add(item)
    METHOD pop(): RETURN items.remove(items.size() - 1)
    METHOD peek(): RETURN items.get(items.size() - 1)
    // no way for callers to violate stack behavior from outside

```

» TIP: If asked to whiteboard a hierarchy, always say the class/interface names out loud, mention access modifiers, and narrate why you chose inheritance vs composition — interviewers weight reasoning as much as syntax.

10. Extra Live-Coding Practice Problems

Q: Reverse a String without using built-in reverse methods.

```
public static String reverse(String s) {
    char[] arr = s.toCharArray();
    int i = 0, j = arr.length - 1;
    while (i < j) {
        char tmp = arr[i]; arr[i] = arr[j]; arr[j] = tmp;
        i++; j--;
    }
    return new String(arr);
}
```

Q: Check if two strings are anagrams.

```
public static boolean isAnagram(String a, String b) {
    if (a.length() != b.length()) return false;
    char[] ca = a.toCharArray(), cb = b.toCharArray();
    Arrays.sort(ca); Arrays.sort(cb);
    return Arrays.equals(ca, cb);
}
```

Q: Find duplicate elements in a list using streams.

```
Set<Integer> seen = new HashSet<>();
Set<Integer> duplicates = list.stream()
    .filter(n -> !seen.add(n))
    .collect(Collectors.toSet());
```

Q: Find the second-highest number in a list using streams.

```
Optional<Integer> secondHighest = list.stream()
    .distinct()
    .sorted(Comparator.reverseOrder())
    .skip(1)
    .findFirst();
```

11. 10-Minute Final Skim (if you're really short on time)

- HashMap internals: hashCode -> bucket, equals for collision match, treeify at 8, resize at 0.75 load factor.
- Streams: filter -> sorted -> collect is your bread-and-butter chain for the live coding task.
- equals()/hashCode() must be overridden together, or hash collections break silently.
- volatile = visibility only. synchronized = visibility + mutual exclusion.
- SOLID, especially Liskov Substitution and Dependency Inversion (ties directly into Spring DI).
- Spring: constructor injection is preferred; @Service/@Repository/@Controller are all @Component under the hood.
- Practice typing the Employee filter+sort+groupBy stream examples by hand, not just reading them.
- Constructor chaining order: superclass constructor always runs before the subclass's own body.
- Overloading = compile-time (same class, different params); Overriding = runtime (subclass, same signature).
- Favor composition over inheritance when the relationship isn't a clean 'is-a'.
- Fail-fast (HashMap/ArrayList) vs fail-safe (ConcurrentHashMap/CopyOnWriteArrayList) iterators.
- Be ready to whiteboard a small class hierarchy (like Shape/Circle/Rectangle) and explain dynamic dispatch.

- Be ready to talk fluently about your own project — expect at least one question tying concepts back to your resume.

Good luck at 4:00 PM — you've got this.